

Embedded systems study plan (3 weeks)

Goal: solid intuition for C internals, low-level/embedded concepts, OS fundamentals, and RTOS basics — culminating in a resume-worthy ESP32 project.

Time budget (flexible):

- Weekdays: 1–2 hrs
- Weekends: ~4 hrs
- Follow your interest — if a topic hooks you, stay in it longer and shift the schedule right rather than forcing the next box.

Each day pairs a short concept study block with a mini hands-on project (1–2 hrs). Concepts stick when you immediately build something with them.

Week 1 — C internals & the machine underneath

Mon–Tue: Bit manipulation (fast, should feel easy given your C background)

- **Study:** bitwise operators, masks, set/clear/toggle/check-bit idioms
- **Resource:** [Bit Twiddling Hacks](#) (reference, skim), any "bitwise operators in C" writeup
- **Mini project:** write a small C library — `set_bit()`, `clear_bit()`, `toggle_bit()`, `check_bit()`, plus a function that packs two values (like your X/Y coords + pen flag) into a byte and unpacks them back. This is literally the encode/decode logic your UART project needs.

Wed–Thu: Memory layout

- **Study:** stack vs. heap vs. static/global (.data/.bss) — where variables actually live and why it matters on memory-constrained targets
- **Resource:** *CS:APP* (Computer Systems: A Programmer's Perspective) — ch. 3 sections on memory, or any "C memory layout" blog writeup if you want something shorter first
- **Mini project:** write a C program that prints the addresses of a local variable, a malloc'd variable, a global variable, and a static variable. Explain in a comment why they land where they do. Deliberately overflow a small stack buffer (safely, just observe the behavior) to *see* stack smashing happen, not just read about it.

Sat (long block, ~4 hrs): Struct packing + memory-mapped I/O concept

- **Study:** struct padding/alignment (why `sizeof` surprises people), then pivot to memory-mapped I/O — the idea that on a microcontroller, "set a pin high" = write to a specific memory address
- **Resource:** search "struct padding C alignment" for a quick writeup; for MMIO, any "memory mapped I/O explained" article — no ESP32 needed yet, the concept is universal
- **Mini project:** simulate a fake "peripheral register" in C — a volatile `uint8_t*` pointing at a byte array standing in for hardware registers, then write functions that set/clear individual bits in it to represent turning a fake LED on/off. This is the *exact* mental model you'll use for real GPIO registers once the ESP32 arrives.

Sun (flex/catch-up day)

- Revisit anything from the week that didn't click. If everything clicked, read ahead into interrupts (next section) or just rest.
-

Week 2 — Systems concepts: interrupts, scheduling, concurrency

Mon–Tue: Interrupts

- **Study:** polling vs. interrupts, what an ISR (interrupt service routine) is, why ISRs should be short
- **Resource:** [search "interrupts vs polling embedded systems"](#) — several good short explainer articles exist
- **Mini project:** no hardware needed yet — write a simulation in C: a "polling" loop that checks a flag repeatedly (representing a hardware event) vs. a signal-handler-based version using C's `signal()` to catch `SIGINT` and treat it like an interrupt firing. Compare the two designs in a short writeup (a few sentences is enough).

Wed–Thu: Processes, threads, scheduling

- **Study:** what a process/thread is, cooperative vs. preemptive scheduling, round-robin as a concrete example
- **Resource:** [OSTEP](#) (free) — the scheduling chapters (start with "Introduction to Scheduling")
- **Mini project:** write a simple round-robin scheduler *simulator* in C — a struct array representing fake "tasks" each with a remaining runtime, and a loop that cycles through giving each a fixed time slice until all are done. Print a timeline of what "ran" when. This is the exact concept an RTOS scheduler implements for real.

Sat (long block, ~4 hrs): Concurrency primitives

- **Study:** race conditions, mutexes, semaphores — why concurrent access to shared data breaks without protection
- **Resource:** OSTEP's concurrency section, specifically the chapters on locks and condition variables
- **Mini project:** write a C program using POSIX threads (pthread) where two threads increment a shared global counter a million times each with no locking — run it, observe the final count is *wrong* (race condition, visible not just theoretical). Then add a `pthread_mutex_t` around the increment and show the count is now correct every run. This single demo is one of the most-cited "aha" moments in systems programming — you'll *feel* why locks exist.

Sun (flex/catch-up day)

Week 3 — RTOS + final project (ESP32 should have arrived)

Mon–Tue: RTOS concepts

- **Study:** how an RTOS differs from a general-purpose OS (hard real-time guarantees, small footprint), core FreeRTOS concepts: tasks, queues, semaphores
- **Resource:** [FreeRTOS official docs](#) — the free "Mastering the FreeRTOS Real Time Kernel" book/guide is unusually well written; read the tasks and queues chapters
- **Mini project (hardware, finally):** flash two independent FreeRTOS tasks onto the ESP32 — one blinks the onboard LED on a timer, the other prints to serial on a different timer. Confirms both your toolchain and your mental model of concurrent tasks running on real hardware.

Wed–Thu: Inter-task communication

- **Study:** FreeRTOS queues and semaphores specifically — how tasks safely pass data to each other
- **Mini project:** two tasks on the ESP32 — one simulates "reading a sensor" (just generates fake values on a timer) and pushes results into a FreeRTOS queue; the other task blocks on the queue and prints whatever it receives. This is the actual pattern you'll use for your UART project (one task receiving bytes, another task rendering to the display).

Fri–Sun (final project — resume piece)

Integrate everything into your UART drawing project, RTOS-ified:

- Task 1: UART receive — reads incoming bytes, decodes your X/Y/pen-flag frame format
 - Task 2: display render — owns the OLED, pulls decoded points off a FreeRTOS queue fed by Task 1, draws them
 - This demonstrates, in one real project: bit-level protocol design, memory-safe inter-task communication, and RTOS task architecture — genuinely resume-worthy because it's not a tutorial clone, it's a protocol *you* designed running on a real scheduler you can explain line by line.
 - Write a short README explaining the architecture (the two-task split, why a queue instead of a shared global, what the frame format is) — this write-up is often what actually gets read in a resume/portfolio context, more than the code itself.
-

Notes on pacing

- If bit manipulation or memory feels too easy in week 1, don't pad it — pull concurrency or RTOS content earlier instead.
- The Saturday/Sunday split assumes weekend = your long block; swap freely if your schedule shifts.
- Every mini project above is deliberately small (1–2 hrs) and buildable without hardware except the week 3 ones — so the plan doesn't stall if the ESP32 shipment slips further.